

"Same Problems.
Better You!"

- ✓ Learn Patterns
- ✓ Solve Problems
- ✓ Crack Interviews

Consistency
Compounds!

DSA

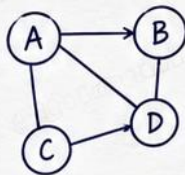
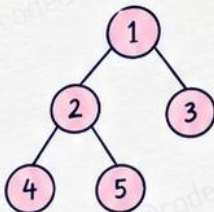
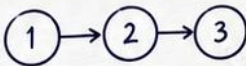
Handwritten Notes

> Complete Guide <

Patterns + Problems

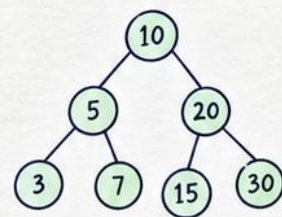
From Basics to Interviews

0	1	2	3
4	8	2	10



- 1 Arrays + Hashing
- 2 Two Pointers
- 3 Sliding Window
- 4 Stack + Queue
- 5 Strings
- 6 Recursion
- 7 Binary Search
- 8 Sorting + Searching
- 9 Matrix
- 10 Linked List Patterns
- 11 Trees + Tree Traversals
- 12 Binary Search Tree (BST)
- 13 Heap + Priority Queue
- 14 Backtracking
- 15 Graphs + BFS + DFS
- 16 Shortest Path + Union Find
- 17 Greedy Algorithms
- 18 Dynamic Programming Patterns
- 19 Intervals + Bit Manipulation + Trie
- 20 DSA Pattern Cheat Sheet + Top Interview Problems

$$\begin{array}{r} 1010 \\ & 1100 \\ \hline 1000 \end{array}$$



Patterns
everywhere!

Code
Learn
Grow
Repeat
😊

"Consistent Practice Creates Confident Developers"

— @coded soul_05 —

1. DSA Fundamentals + Complexity (Patterns)

• Why Learn DSA Patterns ?

- Helps in solving problems faster
- Common patterns repeat in interviews
- Improves problem solving skills
- Builds foundation for advanced topics

"Different problems
Same Patterns!"



• Common DSA Patterns (High Level)

- | | |
|----------------------------------|---|
| 1. Brute Force | 11. Linked List (Fast & Slow, Reversal) |
| 2. Two Pointers | 12. Tree Traversals (DFS / BFS) |
| 3. Sliding Window | 13. Binary Search Tree (BST) |
| 4. Prefix Sum / Difference Array | 14. Heap / Priority Queue |
| 5. Binary Search (on Answer) | 15. Greedy Pattern |
| 6. Sorting + Two Pointers | 16. Divide and Conquer |
| 7. Hashing (Maps / Sets) | 17. Backtracking |
| 8. Frequency Counting | 18. Dynamic Programming (DP) |
| 9. Stack (Monotonic) | 19. Graphs (BFS / DFS / Shortest Path) |
| 10. Queue / Deque | 20. Interval / Range Problems |

• When to Use Which Pattern ?

Pattern	When to Use ?
Brute Force	Small input size, try all possibilities
Two Pointers	Sorted array, pairs, subarrays
Sliding Window	Subarray / substring with conditions
Binary Search	Sorted data, search space, optimize answer
Hashing	Fast lookups, duplicates, frequency
Stack / Queue	Order based problems (next greater, BFS, etc.)
Tree / Graph	Hierarchical or connected data
Greedy	Local optimal choice gives global solution
Dynamic Programming	Overlapping subproblems + optimal substructure
Backtracking	Explore all possibilities (combinations, permutations)

• Key Takeaways

1. Learn and recognize patterns.
2. Practice multiple problems for each pattern.
3. Focus on understanding when to apply a pattern.
4. Most hard problems are just a combination of these patterns.
5. Consistent practice is the key to mastering DSA.

"It's not about remembering solutions, it's about recognizing the pattern." 

2. Arrays + Hashing

Same Concepts
Different Questions
You Got This! 😊

• What is Array ?

An array is a linear data structure that stores elements of the same type in contiguous memory locations. It allows random access using index.

10	20	30	40	50
0	1	2	3	4

• What is Hashing ?

Hashing is a technique to store and retrieve data efficiently using a key-value pair. It provides average $O(1)$ time complexity for insert, delete and search operations.



Common Operations & Complexity

Operation	Array	Hash Map (unordered_map)
Access	$O(1)$	—
Search	$O(n)$	$O(1)$
Insert	$O(n)$	$O(1)$
Delete	$O(n)$	$O(1)$

When to Use ?

Use Case	Data Structure
Need random access	Array
Need to count frequency / check existence	Hash Map / Hash Set
Need ordered data	Array (with sorting)
Need fast lookups	Hash Map / Hash Set

• Common Array Patterns

1. Traversal (find, count, sum, etc.)
2. Find min / max
3. Prefix Sum
4. Two Pointers (on sorted / unsorted array)
5. Sliding Window (subarray problems)
6. In-place modifications
7. Sorting + Searching
8. Frequency Counting (using array / hash map)

• Common Hashing Patterns

1. Frequency count (hash map / hash set)
2. Check if element exists
3. Find first non-repeating element
4. Count pairs with given sum
5. Find subarray with given sum
6. Longest subarray with K distinct elements
7. Use hashing to optimize brute force
8. Two sum / Four sum and variations

Example

1. Frequency of elements using Hash Map

```
arr = [1, 2, 2, 3, 1, 4]
freq = {}
for num in arr:
    if num in freq:
        freq[num] += 1
    else:
        freq[num] = 1
print(freq)
# Output: {1: 2, 2: 2, 3: 1, 4: 1}
```

Hashing
turns many
hard problems
into easy ones 😊

Key Takeaways

1. Arrays are simple but very powerful.
2. Hashing helps in fast lookups and frequency counting.
3. Many array problems can be optimized using hashing.
4. Understand when to use array vs hash map.
5. Practice common patterns, they appear frequently in interviews.

Better Code
Better Opportunities
Keep Going :)

* — @codedsoul_05 — *

• — Page 2 — •

Practice
Build
Improve 😊

Same Concepts
Different Questions
You Got This! 😊

3. Two Pointers

Practice
Solve
Improve
Grow :)

1. What is Two Pointers ?

- Two Pointers is a technique where we use two indices (pointers) to iterate over a data structure, usually an array or a linked list.
- It helps in solving problems more efficiently (usually in $O(n)$ time) instead of using brute force ($O(n^2)$).
- Pointers can move in the same direction, opposite direction or towards each other depending on the problem.
- Mostly used with sorted arrays or when we need to find a pair/subarray/partition.

2. Types of Two Pointers Patterns

- Opposite Direction (start & end)
- Same Direction (fast & slow)
- Sliding Window (variable size window)
- Partitioning / Rearranging
- On Linked List (fast & slow pointers)

3. When to Use ?

- When the problem involves pairs or subarrays.
- When the array is sorted (or can be sorted).
- To reduce time complexity from $O(n^2)$ to $O(n)$.
- When we need to maintain a window or partition.
- Useful in array, string and linked list problems.

4. Common Problems (Two Pointers)

- Find pair with given sum (sorted array)
- Check if array is a palindrome
- Remove duplicates from sorted array
- Move all zeros to end
- Three sum (use two pointers after fixing one)
- Container with most water
- Trapping rain water (variation)
- Linked list cycle detection (Floyd's algorithm)

5. Common Templates

(A) Opposite Direction (start & end)

```
left = 0
right = n - 1
while left < right:
    # process nums[left] and nums[right]
    if condition:
        left += 1
    else:
        right -= 1
```

Used in pair sum, palindrome, container with most water, etc.

(B) Same Direction (fast & slow)

```
slow = 0
for fast in range(n):
    # process nums[fast]
    if condition:
        nums[slow] = nums[fast]
        slow += 1
```

Used in remove duplicates, move zeros, linked list cycle, etc.

(C) Sliding Window (variable size)

```
left = 0
for right in range(n):
    # expand window
    while window condition not valid:
        # shrink window
        left += 1
```

Used in subarray problems, longest substring, etc.

6. Example : Pair with Given Sum (Sorted Array)

```
def twoSumSorted(nums, target):
    left, right = 0, len(nums) - 1
    while left < right:
        s = nums[left] + nums[right]
        if s == target:
            return [left, right]
        elif s < target:
            left += 1
        else:
            right -= 1
    return []
```

Dry Run:

nums = [1, 2, 4, 7, 11]
target = 9

left	right
↓	↓
1	11

1 + 11 = 12 (> 9) → right--

1 + 7 = 8 (< 9) → left++

2 + 7 = 9 → return [1, 3]

7. Key Takeaways

- Two Pointers can reduce time complexity to $O(n)$.
- Identify the type of two pointer pattern.
- Use sorted array when possible.
- Practice variations to build intuition.
- Very common in interviews.

Same pattern appears in many problems, just the logic changes! 😊

Better Code
Better Opportunities
Keep Going :)

* — @coded soul_05 — *

Practice
Build
Improve 😊

4. Sliding Window

- What is Sliding Window ?

Sliding Window is a technique used to solve problems on arrays or strings where we need to find a subarray or substring that satisfies a given condition. Instead of using nested loops ($O(n^2)$), we maintain a window and slide it over the data to get an optimized solution (usually $O(n)$).

- Types of Sliding Window

1. Fixed Size Window

- When the size of the window is constant
- Eg. maximum sum of subarray of size k

2. Variable Size Window

- When the size of the window changes based on a condition
- Eg. longest substring without repeating characters

- When to Use Sliding Window ?

- Problems involving subarrays or substrings
- Need to find maximum / minimum / count in a window
- To optimize from $O(n^2)$ to $O(n)$
- Works well with arrays and strings

- Example (Fixed Size Window)

```
// Maximum sum subarray of size k
int maxSum(int[] nums, int k) {
    int n = nums.length;
    int sum = 0;
    for (int i = 0; i < k; i++) sum += nums[i];
    int ans = sum;
    for (int i = k; i < n; i++) {
        sum += nums[i] - nums[i - k];
        ans = Math.max(ans, sum);
    }
    return ans;
}
```

- Common Problems (Sliding Window)

1. Maximum sum subarray of size k
2. Longest substring without repeating characters
3. Longest substring with at most k distinct characters
4. Minimum size subarray sum ($\geq$ target)
5. Count subarrays with given sum
6. Find all anagrams in a string
7. Smallest window containing all characters
8. Longest repeating character replacement

- General Template (Fixed Size)

```
int n = nums.length;
int k = ...;

// process first window
for (int i = 0; i < k; i++) {
    // add nums[i]
}

// slide the window
for (int i = k; i < n; i++) {
    // add nums[i]
    // remove nums[i - k]
    // update answer
}
```

- General Template (Variable Size)

```
int left = 0;
for (int right = 0; right < n; right++) {
    // add nums[right]
    // shrink window while condition not valid
    while (/* condition */) {
        // remove nums[left]
        left++;
    }
    // update answer
}
```

Note :

Sliding Window helps us optimize brute force solutions by reusing previous computations instead of recalculating everything.

5. Prefix Sum + Difference Array

• What is Prefix Sum ?

Prefix Sum is a technique where we precompute the cumulative sum of an array, so that we can find the sum of any subarray in $O(1)$ time instead of $O(n)$.

• Idea

Create an array `prefix[]` where

```
prefix[i] = prefix[i - 1] + nums[i]
prefix[0] = nums[0]
```

• Example

`nums = [1, 2, 3, 4, 5]`

Index (i)	0	1	2	3	4
nums[i]	1	2	3	4	5
prefix[i]	1	3	6	10	15

Sum from l to r = `prefix[r] - prefix[l - 1]`
(if $l > 0$)

• Code (Prefix Sum)

```
int n = nums.length;
int[] prefix = new int[n];
prefix[0] = nums[0];
for (int i = 1; i < n; i++) {
    prefix[i] = prefix[i - 1] + nums[i];
}

// sum of subarray [l, r]
int sum = prefix[r] - (l > 0 ? prefix[l - 1] : 0);
```

• When to Use Prefix Sum ?

- Find sum of subarrays quickly
- Count subarrays with given sum
- Useful in problems on ranges, frequencies, and cumulative calculations

• What is Difference Array ?

Difference Array is a technique used when we need to perform multiple range updates efficiently. Instead of updating each element in the range, we update only two positions and then compute the final array using prefix sum.

• Idea

For a range update $[l, r]$ with value `val`:

```
diff[l] += val
diff[r + 1] -= val (if r + 1 < n)
```

• Example

$n = 5$, operations:

1. add 3 to range $[1, 3]$
2. add 2 to range $[0, 2]$

Initial `diff` = `[0, 0, 0, 0, 0]` ($n + 1$ size)

Operation	diff array ($n + 1 = 6$)
Initial	<code>[0, 0, 0, 0, 0, 0]</code>
After $[1, 3] + 3$	<code>[0, 3, 0, 0, -3, 0]</code>
After $[0, 2] + 2$	<code>[2, 3, 0, -2, -3, 0]</code>

Now take prefix sum of `diff`:

Resultant array = `[2, 5, 5, 3, 0]`

• Code (Difference Array)

```
int n = 5;
int[] diff = new int[n + 1];
// range update [l, r] with val
diff[l] += val;
diff[r + 1] -= val; // if r + 1 < n

// build final array using prefix sum
int[] res = new int[n];
res[0] = diff[0];
for (int i = 1; i < n; i++) {
    res[i] = res[i - 1] + diff[i];
}
```

• When to Use Difference Array ?

- Multiple range updates
- When we need to add a value to all elements in a range
- Useful in problems like booking systems, frequency updates, etc.

6. Binary Search

• What is Binary Search ?

Binary Search is a technique to find a target element in a sorted array by repeatedly dividing the search space into half. It works in $O(\log n)$ time by comparing the middle element and discarding half of the search space each time.

• Template (Iterative)

```
int binarySearch(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1; // not found
}
```

• Template (Recursive)

```
int binarySearch(int[] nums, int target, int left, int right) {
    if (left > right) return -1;
    int mid = left + (right - left) / 2;
    if (nums[mid] == target) return mid;
    else if (nums[mid] < target)
        return binarySearch(nums, target, mid + 1, right);
    else
        return binarySearch(nums, target, left, mid - 1);
}
```

• When to Use ?

- When the data is sorted
- To find a specific element (exists or not)
- To find the first/last occurrence
- To find insertion position
- To search on the answer space (binary search on range)
- Useful in optimization problems

• Common Binary Search Patterns

1. Find element in sorted array
2. Find first occurrence of target
3. Find last occurrence of target
4. Find number of occurrences
5. Find insertion position
6. Search in rotated sorted array
7. Binary search on answer (search space)
8. Find minimum/maximum in a sorted range

• Variations

1. First Occurrence (lower bound)
2. Last Occurrence (upper bound)
3. Search in Rotated Sorted Array
4. Find Peak Element
5. Find Minimum in Rotated Sorted Array
6. Binary Search on Answer (e.g. minimize max, maximize minimum)
7. Find square root (using binary search)
8. Find nth root, etc.

• Example : First Occurrence

```
int firstOccurrence(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    int ans = -1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) {
            ans = mid; // store and go left
            right = mid - 1;
        } else if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return ans;
}
```

• Time & Space Complexity

Operation	Complexity
Time	$O(\log n)$
Space	$O(1)$ (iterative) / $O(\log n)$ (recursive)

Note :

Binary Search works only on sorted data or monotonic functions.

7. Sorting Patterns

• What is Sorting?

Sorting means arranging elements in a particular order (usually ascending or descending). It helps in searching, finding patterns and optimizing solutions.

• Why Sorting ?

- Helps in finding duplicates
- Useful in two pointer problems
- Required in greedy algorithms
- Makes binary search possible
- Helps in range and interval problems

• Types of Sorting

1. Built-in Sort (library function)
2. Custom Comparator Sorting
3. Sorting in Descending Order
4. Sorting based on a specific property (e.g. by second element, by length, etc.)

• Basic Syntax (C++)

```
#include <bits/stdc++.h>
using namespace std;

vector<int> v = {5, 1, 4, 2, 3};

// sort in ascending order
sort(v.begin(), v.end());

// sort in descending order
sort(v.begin(), v.end(), greater<int>());
```

• When to Use Sorting ?

- When order of elements matters
- To find duplicates
- To use two pointer technique
- In greedy and interval problems
- When combined with binary search
- To process data efficiently

• Common Sorting Patterns

1. Sort array in ascending / descending order
2. Custom comparator sorting (e.g. pair, struct)
3. Sort by multiple criteria (e.g. first then second)
4. Sort strings (lexicographical)
5. Sort based on frequency / value / length
6. Sort and then apply two pointers
7. Sort intervals (by start time)
8. Sort with custom logic (lambda / comparator)

• Examples

1. Sort pairs by second element

```
vector<pair<int,int>> v = {{1, 4}, {2, 1}, {3, 3}};
sort(v.begin(), v.end(), [](auto &a, auto &b) {
    return a.second < b.second;
});
```

2. Sort strings by length

```
vector<string> v = {"apple", "cat", "banana"};
sort(v.begin(), v.end(), [](string &a, string &b) {
    return a.length() < b.length();
});
```

3. Sort in descending order

```
vector<int> v = {5, 1, 4, 2, 3};
sort(v.begin(), v.end(), greater<int>());
// v = {5, 4, 3, 2, 1}
```

• Time & Space Complexity

Algorithm	Time Complexity	Space Complexity
Built-in sort (C++ STL)	$O(n \log n)$	$O(1)$
Custom comparator sort	$O(n \log n)$	$O(1)$
Stable sort	$O(n \log n)$	$O(n)$

Note :

Sorting is a powerful tool. Many complex problems become simple after sorting !

8. Stack + Monotonic Stack

• What is Stack ?

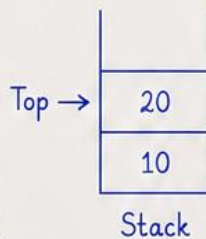
Stack is a linear data structure that follows LIFO (Last In First Out) order.
The last element added is the first one to be removed.

• Basic Operations

Push() - Add an element to top
Pop() - Remove top element
Top() - Get top element
Empty() - Check if stack is empty

• Example

```
stack<int> st;
st.push(10);
st.push(20);
cout << st.top(); // 20
st.pop();
cout << st.top(); // 10
```



• When to Use Stack ?

- When we need LIFO order
- In problems with nested structures (parentheses, brackets)
- Undo / Redo functionality
- Expression evaluation (infix, postfix, prefix)
- Used in many monotonic stack problems

• Common Problems (Stack)

1. Valid Parentheses
2. Next Greater Element
3. Next Smaller Element
4. Stock Span Problem
5. Largest Rectangle in Histogram
6. Evaluate Reverse Polish Notation
7. Remove K Digits
8. Implement Queue using Stacks

• What is Monotonic Stack ?

Monotonic Stack is a stack that maintains its elements in either increasing or decreasing order.
It helps to find the next/previous greater or smaller element in $O(n)$ time.

• Types of Monotonic Stack

1. Increasing Stack - elements are in increasing order (used to find next smaller element)
2. Decreasing Stack - elements are in decreasing order (used to find next greater element)

• Example : Next Greater Element (Monotonic Stack)

```
vector<int> nextGreater(vector<int>& nums) {
    int n = nums.size();
    vector<int> res(n, -1);
    stack<int> st;
    for (int i = 0; i < n; i++) {
        while (!st.empty() && nums[st.top()] < nums[i]) {
            res[st.top()] = nums[i];
            st.pop();
        }
        st.push(i);
    }
    return res;
}
```

Dry Run (nums = [2, 1, 2, 4, 3])

i	nums[i]	Stack (indices)	res
0	2	[0]	[-1, -1, -1, -1, -1]
1	1	[0, 1]	[-1, -1, -1, -1, -1]
2	2	[2]	[2, -1, -1, -1, -1]
3	4	[3]	[2, 2, -1, -1, -1]
4	3	[3, 4]	[2, 2, -1, -1, -1]

Note :

Monotonic Stack helps us solve many problems in $O(n)$ time which are hard to solve using brute force.

9. Queue + Deque

• What is Queue ?

Queue is a linear data structure that follows FIFO (First In First Out) order.
The element inserted first is removed first.

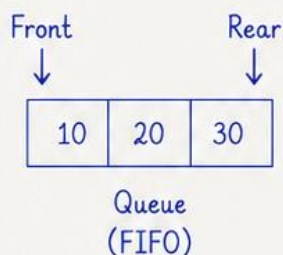
• Basic Operations (Queue)

enqueue()	- Add an element to the rear
dequeue()	- Remove an element from front
front()	- Get the front element
back()	- Get the rear element
size()	- Number of elements
empty()	- Check if queue is empty

• Example (Queue)

```
queue<int> q;
q.push(10);
q.push(20);
q.push(30);

cout << q.front(); // 10
q.pop();
cout << q.front(); // 20
```



• Common Problems (Queue)

1. Implement Queue using arrays / linked list
2. First non-repeating character in a stream
3. Sliding window maximum (using deque)
4. Rotten oranges (BFS)
5. Number of islands (BFS)
6. Task scheduling / CPU scheduling
7. Print first k elements of a queue
8. Implement a circular queue

Note :

Queue is widely used in BFS, scheduling problems and in handling data in FIFO order.

• What is Deque ?

Deque (Double Ended Queue) is a linear data structure that allows insertion and deletion from both ends (front and rear).
It can be used as a queue, a stack, or both.

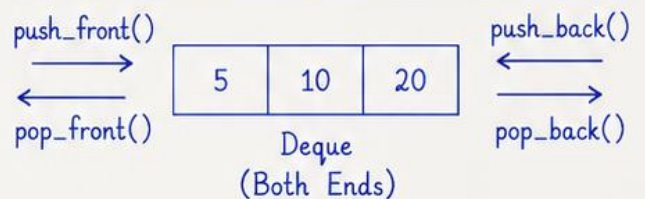
• Basic Operations (Deque)

push_front()	- Insert at front
push_back()	- Insert at rear
pop_front()	- Delete from front
pop_back()	- Delete from rear
front()	- Get front element
back()	- Get rear element
empty()	- Check if deque is empty
size()	- Number of elements

• Example (Deque)

```
deque<int> dq;
dq.push_back(10); // 10
dq.push_back(20); // 10 20
dq.push_front(5); // 5 10 20

cout << dq.front(); // 5
cout << dq.back(); // 20
dq.pop_front(); // 10 20
dq.pop_back(); // 10
```



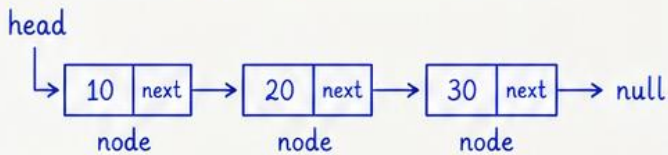
• Common Problems (Deque)

1. Sliding window maximum / minimum
2. LRU Cache
3. First negative number in every window of size k
4. Maximum of minimum for every window size
5. Palindrome check (using deque)
6. Implement stack using deque
7. Implement queue using deque

10. Linked List Patterns

• What is a Linked List ?

A Linked List is a linear data structure where elements (nodes) are stored in non-contiguous memory locations. Each node contains data and a reference (pointer) to the next node.



• Types of Linked List

1. Singly Linked List (each node points to next)
2. Doubly Linked List (points to both next and previous)
3. Circular Linked List (last node points to head)

• Basic Node Structure (C++)

```
struct Node {
    int val;
    Node* next;
    Node(int x) {
        val = x;
        next = nullptr;
    }
};
```

• Common Operations

Operation	Time Complexity
Insert at beginning	O(1)
Insert at end	O(n)
Delete a node	O(n)
Search a value	O(n)
Traverse	O(n)

• Why Linked List ?

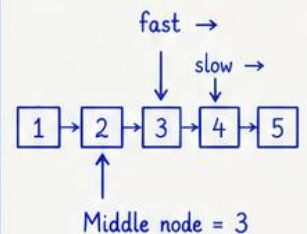
- Dynamic size (no need to specify size)
- Efficient insertions and deletions
- Useful in implementing stacks, queues, hash tables
- Used in many real world applications (LRU Cache, Undo/Redo, etc.)

• Common Linked List Patterns

1. Traverse the list
2. Find length of linked list
3. Find middle node (Slow & Fast pointer)
4. Reverse a linked list
5. Detect cycle (Floyd's algorithm)
6. Find start of cycle
7. Merge two sorted linked lists
8. Remove Nth node from end
9. Reorder list ($L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \dots$)
10. Check if linked list is palindrome

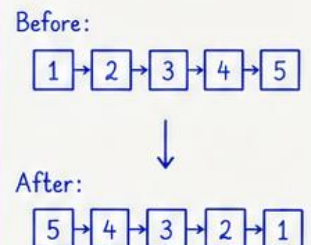
• Pattern 1: Find Middle Node (Slow & Fast Pointer)

```
Node* middleNode(Node* head) {
    Node* slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```



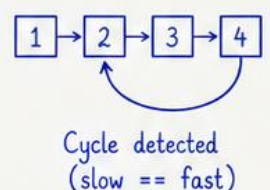
• Pattern 2: Reverse a Linked List

```
Node* reverseList(Node* head) {
    Node* prev = nullptr;
    Node* curr = head;
    while (curr) {
        Node* next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```



• Pattern 3: Detect Cycle (Floyd's Algorithm)

```
bool hasCycle(Node* head) {
    Node* slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
```



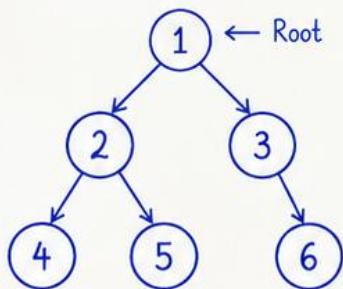
Note :

- Linked list problems are very common in interviews.
- Practice multiple patterns to build intuition.
- Most problems can be solved using simple pointer techniques.

11. Trees + Tree Traversals

• What is a Tree ?

A Tree is a non-linear data structure that consists of nodes connected by edges. It has a hierarchical structure with a single root node and zero or more child nodes. There are no cycles in a tree.



- Each node has at most one parent (except root).
- A node can have zero or more children.
- Total nodes = n
Total edges = $n - 1$

• Types of Trees

1. Binary Tree (each node has at most 2 children)
2. Binary Search Tree (left < root < right)
3. Balanced Binary Tree (height is $O(\log n)$)
4. Full Binary Tree (every node has 0 or 2 children)
5. Complete Binary Tree (all levels filled except last)
6. Perfect Binary Tree (all levels completely filled)
7. N-ary Tree (each node can have N children)

• Basic Node Structure (C#)

```
public class TreeNode
{
    public int val;
    public TreeNode left;
    public TreeNode right;

    public TreeNode(int val)
    {
        this.val = val;
        this.left = null;
        this.right = null;
    }
}
```

• Why Trees ?

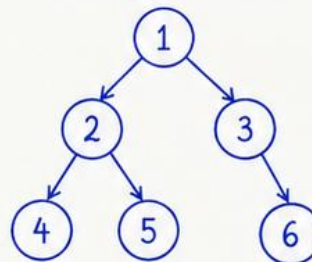
- Hierarchical data representation
- Used in searching, sorting, indexing (e.g. BST)
- Used in file systems, databases, compilers
- Useful in many real world applications
- Basis for advanced data structures (Heap, Trie, etc.)

• Tree Traversals

Tree traversal means visiting all the nodes of a tree in a specific order.

Traversal	Order	Use Case
Preorder (NLR)	Root → Left → Right	Copy tree, prefix expression
Inorder (LNR)	Left → Root → Right	BST gives sorted order
Postorder (LRN)	Left → Right → Root	Delete tree, postfix expression
Level Order	Level by level (BFS)	Find level, minimum depth, etc.

• Example Tree



Traversals for this tree:

Preorder : 1 2 4 5 3 6
Inorder : 4 2 5 1 3 6
Postorder : 4 5 2 6 3 1
Level Order: 1 2 3 4 5 6

• Traversal Code (C#)

1. Preorder (NLR)

```
void Preorder(TreeNode root) {
    if (root == null) return;
    Console.Write(root.val + " ");
    Preorder(root.left);
    Preorder(root.right);
}
```

2. Inorder (LNR)

```
void Inorder(TreeNode root) {
    if (root == null) return;
    Inorder(root.left);
    Console.Write(root.val + " ");
    Inorder(root.right);
}
```

3. Postorder (LRN)

```
void Postorder(TreeNode root) {
    if (root == null) return;
    Postorder(root.left);
    Postorder(root.right);
    Console.Write(root.val + " ");
}
```

4. Level Order (BFS)

```
void LevelOrder(TreeNode root) {
    if (root == null) return;
    Queue<TreeNode> q = new();
    q.Enqueue(root);
    while (q.Count > 0) {
        var node = q.Dequeue();
        Console.Write(node.val + " ");
        if (node.left != null) q.Enqueue(node.left);
        if (node.right != null) q.Enqueue(node.right);
    }
}
```

Note :

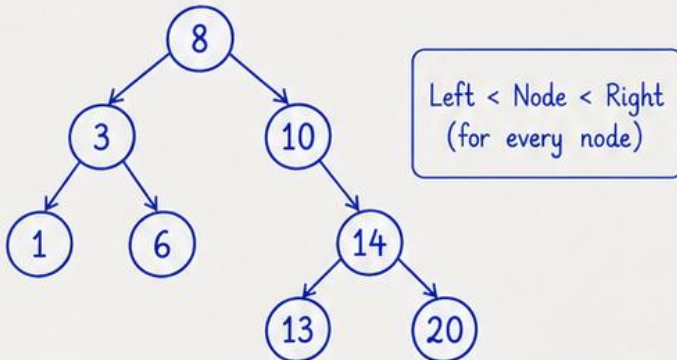
- Inorder traversal of a BST gives sorted elements.
- Tree problems are very common in interviews.
- Master recursion and think about base case always.
- Try solving both recursive and iterative solutions.

12. Binary Search Tree (BST)

• What is a Binary Search Tree ?

A Binary Search Tree (BST) is a binary tree where for every node:

- all nodes in the left subtree have smaller values,
 - all nodes in the right subtree have greater values.
- It allows efficient search, insert and delete operations with average time complexity $O(\log n)$.



• Basic Operations

Operation	Average Time Complexity
Search	$O(\log n)$
Insert	$O(\log n)$
Delete	$O(\log n)$
Find Min / Max	$O(\log n)$
Inorder Traversal	$O(n)$

• Node Structure (C#)

```
public class TreeNode
{
    public int val;
    public TreeNode left;
    public TreeNode right;

    public TreeNode(int val)
    {
        this.val = val;
        this.left = null;
        this.right = null;
    }
}
```

• Why BST ?

- Provides fast search, insert and delete ($O(\log n)$)
- Maintains elements in sorted order (inorder traversal)
- Useful in databases, compilers, file systems
- Basis for advanced data structures (AVL, Red-Black Tree, etc.)

• Common BST Operations (C#)

1. Search in BST

```
bool Search(TreeNode root, int key)
{
    if (root == null) return false;
    if (key == root.val) return true;
    if (key < root.val) return Search(root.left, key);
    return Search(root.right, key);
}
```

2. Insert in BST

```
TreeNode Insert(TreeNode root, int key)
{
    if (root == null) return new TreeNode(key);
    if (key < root.val) root.left = Insert(root.left, key);
    else if (key > root.val) root.right = Insert(root.right, key);
    return root;
}
```

3. Delete in BST

```
TreeNode Delete(TreeNode root, int key)
{
    if (root == null) return null;
    if (key < root.val) root.left = Delete(root.left, key);
    else if (key > root.val) root.right = Delete(root.right, key);
    else
    {
        // Node to be deleted
        if (root.left == null) return root.right;
        if (root.right == null) return root.left;
        // Node with two children: replace with inorder successor
        TreeNode minNode = GetMin(root.right);
        root.val = minNode.val;
        root.right = Delete(root.right, minNode.val);
    }
    return root;
}

TreeNode GetMin(TreeNode node)
{
    while (node.left != null) node = node.left;
    return node;
}
```

• Tree Traversals in BST

Traversal	Order	Result (for above tree)
Inorder (LNR)	Left → Node → Right	1 3 6 8 10 13 14 20
Preorder (NLR)	Node → Left → Right	8 3 1 6 10 14 13 20
Postorder (LRN)	Left → Right → Node	1 6 3 13 20 14 10 8
Level Order	Level by level (BFS)	8 3 10 1 6 14 13 20

Note :

Inorder traversal of a BST always gives elements in sorted (ascending) order.

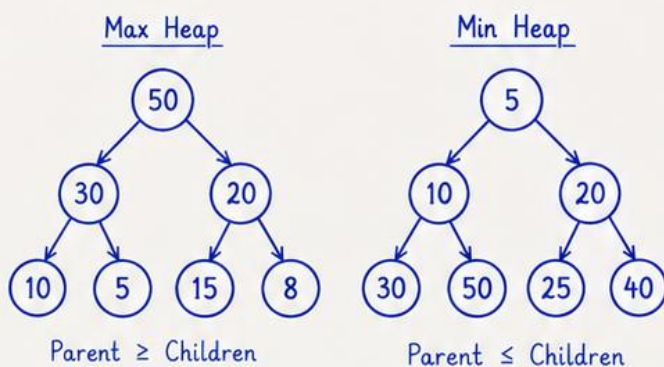
13. Heap + Priority Queue

• What is a Heap ?

A Heap is a complete binary tree that satisfies the heap property.

- Max Heap: parent $\geq$ children (largest at root)
- Min Heap: parent $\leq$ children (smallest at root)

Used to efficiently get the minimum or maximum element in $O(1)$ time.



• What is a Priority Queue ?

A Priority Queue is a data structure that processes elements based on their priority (not FIFO).

- Implemented using a heap.
- By default, C# uses a Min Heap.
- Elements with higher priority (smaller value) are dequeued first.

• Basic Operations

Operation	Time Complexity	Description
Insert / Enqueue	$O(\log n)$	Add element
Delete / Dequeue	$O(\log n)$	Remove top element
Peek	$O(1)$	Get top element
Build Heap	$O(n)$	Create heap from array

• Why Heap / Priority Queue ?

- Efficient way to get min or max element
- Used in scheduling, shortest path (Dijkstra), Huffman coding, k closest elements, etc.
- Helps solve problems with ordering and priority
- More efficient than sorting when we only need top k elements

• C# PriorityQueue<TElement, TPriority>

C# provides a built-in PriorityQueue in System.Collections.Generic which is a min priority queue.

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        var pq = new PriorityQueue<string, int>();

        pq.Enqueue("A", 3);
        pq.Enqueue("B", 1);
        pq.Enqueue("C", 2);

        Console.WriteLine(pq.Dequeue()); // B (1)
        Console.WriteLine(pq.Dequeue()); // C (2)
        Console.WriteLine(pq.Dequeue()); // A (3)
    }
}
```

• Example: K Largest Elements (using Min Heap)

```
class Program
{
    static List<int> Klargest(int[] nums, int k)
    {
        var pq = new PriorityQueue<int, int>(); // min heap
        foreach (int num in nums)
        {
            pq.Enqueue(num, num);
            if (pq.Count > k)
                pq.Dequeue();
        }
        return pq.UnorderedItems.Select(x => x.Element).ToList();
    }
}
```

• Common Problems

1. K^{th} largest / smallest element
2. Top K frequent elements
3. Merge K sorted lists
4. Task scheduling (CPU scheduling)
5. Dijkstra's shortest path
6. Median in a data stream
7. K closest points to origin
8. Huffman coding (data compression)

Note :

- C# PriorityQueue is a Min Heap by default.
- For Max Heap, use negative values or custom comparer.

14. Backtracking

• What is Backtracking ?

Backtracking is a problem solving technique where we try all possible choices and undo (backtrack) a choice if it does not lead to a valid solution. It is basically DFS with a "try → undo" approach.

"Explore → If not valid → Backtrack → Try next"

• General Template

```
void backtrack(...) {  
    if (base condition) {  
        // store the answer  
        return;  
    }  
  
    for (each choice) {  
        // make choice  
        // explore  
        backtrack(...);  
        // undo choice (backtrack)  
    }  
}
```

1. Make a choice
2. Explore further
3. Undo the choice
4. Try next choice

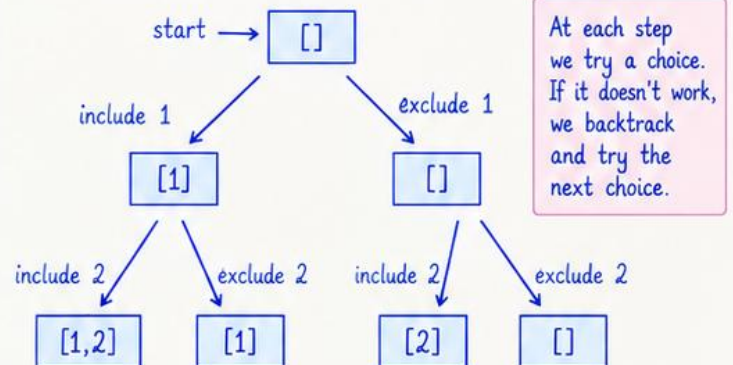
• Example 1: Generate All Subsets

```
void subsets(int i, vector<int>& nums, vector<int>& path,  
            vector<vector<int>>& res) {  
    if (i == nums.size()) {  
        res.push_back(path);  
        return;  
    }  
  
    // Choice 1: include nums[i]  
    path.push_back(nums[i]);  
    subsets(i + 1, nums, path, res);  
    path.pop_back(); // backtrack  
  
    // Choice 2: exclude nums[i]  
    subsets(i + 1, nums, path, res);  
}
```

• Why Backtracking ?

1. Useful for problems with multiple possible solutions
2. Helps explore all combinations / permutations
3. Simple to implement (recursive)
4. Used in many real world scenarios (scheduling, puzzles, etc.)

• Recursion Tree (Subsets Example)



• Common Backtracking Problems

1. Generate all subsets
2. Generate all permutations
3. Combination Sum (I, II)
4. N-Queens Problem
5. Sudoku Solver
6. Rat in a Maze
7. Word Search
8. Partition into K subsets
9. Generate valid parentheses
10. Solve puzzles (e.g. cryptarithmic)

• Example 2: N-Queens

```
void solve(int col, vector<string>& board, vector<vector<string>>& res) {  
    int n = board.size();  
    if (col == n) {  
        res.push_back(board);  
        return;  
    }  
    for (int row = 0; row < n; row++) {  
        if (isSafe(row, col, board)) {  
            board[row][col] = 'Q'; // make choice  
            solve(col + 1, board, res); // explore  
            board[row][col] = '.'; // backtrack (undo)  
        }  
    }  
}
```

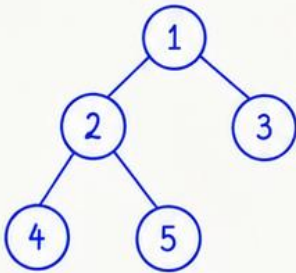
Note :

- Backtracking is basically DFS with an undo step.
- Think in terms of choices and constraints.
- Use pruning to avoid unnecessary exploration.
- Always backtrack to restore the previous state.

15. Graphs + BFS + DFS

• What is a Graph ?

A graph is a non-linear data structure consisting of a set of nodes (vertices) and a set of edges that connect pairs of nodes. It can be either directed or undirected and can have weights on edges.



- Nodes (V) = vertices
- Edges (E) = connections
- Can be directed / undirected
- Can be weighted / unweighted
- Common representations: Adjacency List, Adjacency Matrix

• Types of Graphs

Type	Description
Undirected Graph	Edges have no direction (A - B)
Directed Graph (DiGraph)	Edges have direction (A → B)
Weighted Graph	Edges have weights/cost
Unweighted Graph	All edges have equal weight
Cyclic Graph	Contains a cycle
Acyclic Graph	Does not contain a cycle (DAG)
Connected Graph	All nodes are reachable
Disconnected Graph	Some nodes are not reachable

• Graph Representation (C#)

```
// Adjacency List
int n = 5;
List<int>[] graph = new List<int>[n];
for (int i = 0; i < n; i++)
    graph[i] = new List<int>();

// Undirected edge u - v
graph[u].Add(v);
graph[v].Add(u);

// For weighted graph (using tuple)
List<(int to, int weight)>[] wGraph = new List<(int, int)>[n];
```

• Why Graphs ?

- Model real world relationships (social networks, maps, etc.)
- Used in path finding, connectivity, scheduling, networking
- Basis for many advanced algorithms (shortest path, MST, etc.)

• Breadth First Search (BFS)

BFS explores nodes level by level using a queue. It finds the shortest path (in unweighted graphs) from a source node to all other nodes.

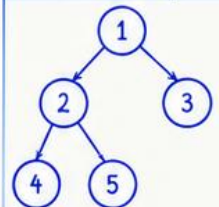
```
void BFS(int start, List<int>[] graph) {
    bool[] visited = new bool[graph.Length];
    Queue<int> q = new Queue<int>();
    q.Enqueue(start);
    visited[start] = true;

    while (q.Count > 0) {
        int node = q.Dequeue();
        Console.WriteLine(node + " "); // process node
        foreach (int nei in graph[node]) {
            if (!visited[nei]) {
                visited[nei] = true;
                q.Enqueue(nei);
            }
        }
    }
}
```

Time Complexity: $O(V + E)$

Use Case: Shortest path, level order, connected components

BFS Example



Start from 1

BFS Order:

1 2 3 4 5

• Depth First Search (DFS)

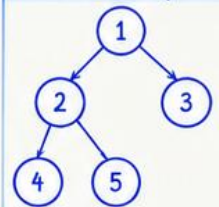
DFS explores as deep as possible along a branch before backtracking. It can be implemented using recursion or a stack.

```
void DFS(int node, List<int>[] graph, bool[] visited) {
    visited[node] = true;
    Console.WriteLine(node + " "); // process node
    foreach (int nei in graph[node]) {
        if (!visited[nei]) {
            DFS(nei, graph, visited);
        }
    }
}
```

Time Complexity: $O(V + E)$

Use Case: Cycle detection, topological sort, connected components, backtracking problems

DFS Example



Start from 1

DFS Order (one possible):

1 2 4 5 3

• Common Graph Problems

1. Number of connected components
2. Detect cycle in an undirected / directed graph
3. Find shortest path (BFS / Dijkstra)
4. Topological sort (for DAG)
5. Minimum Spanning Tree (Prim's / Kruskal's)
6. Flood fill / number of islands
7. Clone a graph
8. Find if a path exists between two nodes
9. Strongly connected components (Kosaraju / Tarjan)
10. Bipartite graph check

Note :

- Use BFS for shortest path in unweighted graphs.
- Use DFS for exploring, cycle detection, and topological sort.
- Choose the right representation based on the problem.

16. Shortest Path + Union Find

• Shortest Path

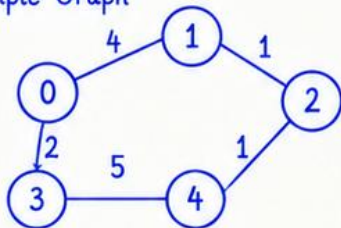
• What is Shortest Path ?

Shortest path means finding the minimum cost/distance from a source node to a target node in a graph. It can be used in many real world problems like: navigation, network routing, social networks, etc.

• Types of Shortest Path Problems

Algorithm	When to Use	Time Complexity
BFS	Unweighted graph (all edges = 1)	$O(V + E)$
Dijkstra	Weighted graph (all weights ≥ 0)	$O((V + E) \log V)$
Bellman-Ford	Graphs with negative weights	$O(V \times E)$
Floyd-Warshall	All pairs shortest path	$O(V^3)$

• Example Graph



Find shortest path from node 0 to 4 (using Dijkstra).

• Dijkstra's Algorithm (C#)

```
int[] Dijkstra(int n, List<(int to, int wt)>[] graph, int src)
{
    int[] dist = Enumerable.Repeat(int.MaxValue, n).ToArray();
    dist[src] = 0;
    var pq = new PriorityQueue<(int node, int dist), int>();
    pq.Enqueue((src, 0), 0);
    while (pq.Count > 0)
    {
        var (node, d) = pq.Dequeue();
        if (d > dist[node]) continue;
        foreach (var (nei, wt) in graph[node])
        {
            if (dist[nei] > d + wt)
            {
                dist[nei] = d + wt;
                pq.Enqueue((nei, dist[nei]), dist[nei]);
            }
        }
    }
    return dist;
}
```

Use a min heap to always pick the closest node first.

• Key Points

- Use BFS for unweighted graphs.
- Use Dijkstra for non-negative weighted graphs.
- Use Bellman-Ford if there are negative weights.
- Use Floyd-Warshall for all pairs shortest paths.

• Union Find (Disjoint Set Union)

• What is Union Find ?

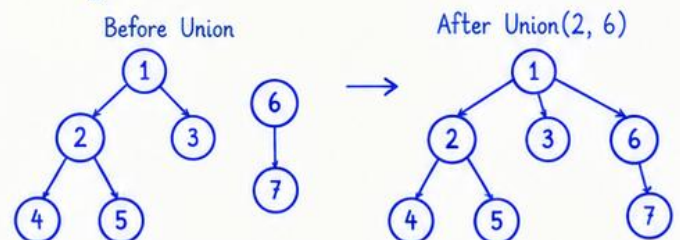
Union Find (DSU) is a data structure that keeps track of disjoint sets and supports two main operations:

- Find(x) $\rightarrow$ returns the representative (root) of the set.
- Union(x, y) $\rightarrow$ merges two sets.

Used in problems like connected components, cycle detection, Kruskal's MST, etc.

• Optimizations

- Path Compression – while finding root, directly attach node to the root (makes tree flat).
- Union by Rank / Size – always attach smaller tree under larger tree.



• Union Find Implementation (C#)

```
class UnionFind
{
    int[] parent, rank;
    public UnionFind(int n)
    {
        parent = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    public int Find(int x)
    {
        if (parent[x] != x)
            parent[x] = Find(parent[x]); // Path Compression
        return parent[x];
    }
    public void Union(int x, int y)
    {
        int rootX = Find(x);
        int rootY = Find(y);
        if (rootX == rootY) return;
        if (rank[rootX] < rank[rootY]) parent[rootX] = rootY;
        else if (rank[rootX] > rank[rootY]) parent[rootY] = rootX;
        else { parent[rootY] = rootX; rank[rootX]++; }
    }
}
```

Find $\rightarrow$ with path compression
Union $\rightarrow$ with rank (or size)

• Common Problems Using Union Find

1. Number of connected components
2. Detect cycle in an undirected graph
3. Kruskal's Minimum Spanning Tree (MST)
4. Accounts Merge
5. Similar String Groups
6. Satisfiability of Equations
7. Number of Islands (2D grid)
8. Redundant Connection

Note :

- Union Find works very efficiently with near $O(1)$ per operation.
- Used a lot in graph and grid problems.

17. Greedy Algorithms

• What is Greedy Algorithm ?

A Greedy algorithm makes the locally optimal choice at each step with the hope of finding the global optimum. It does not backtrack and does not consider all possibilities.

“Choose the best option available right now.”
→ Simple, fast and works for many problems.

• When to Use Greedy ?

1. When problem has Optimal Substructure (optimal solution can be built from optimal subproblems)
2. When Greedy Choice Property holds (local optimal choice leads to global optimal solution)
3. Usually for optimization problems

• Common Greedy Problems

1. Activity Selection
2. Fractional Knapsack
3. Huffman Coding
4. Minimum Number of Platforms
5. Job Sequencing
6. Coin Change (minimum coins - for specific coins)
7. Meeting Rooms / Interval Scheduling
8. Gas Station
9. Minimum Spanning Tree (Kruskal's)
10. Dijkstra's Shortest Path (uses greedy idea)
11. Merge Intervals
12. Assign Cookies, etc.

Not all problems can be solved using greedy. We must prove that the greedy choice is correct.

• Example 1: Activity Selection Problem

Given start and end times of activities, select the maximum number of non-overlapping activities.

Approach: Always pick the activity that ends the earliest.

Activity	Start	End
A1	1	3
A2	2	5
A3	4	7
A4	6	9
A5	8	10

Selected Activities:

A1 (1,3)
A3 (4,7)
A5 (8,10)

Maximum = 3

• Example 2: Fractional Knapsack

Given n items with value and weight, and a knapsack of capacity W . Maximize value (you can take fractions).

Approach: Pick items with highest value/weight ratio first.

Item	Value	Weight	Value/Weight
1	60	10	6.0
2	100	20	5.0
3	120	30	4.0

Capacity = 50
Take item 1 (10)
Take item 2 (20)
Take 2/3 of item 3 (20)

Total Value = $60 + 100 + (2/3)*120 = 240$

• Example 3: Coin Change (Min Coins)

Given coins of certain denominations, find the minimum number of coins to make amount (works when coins are canonical like 1, 2, 5, 10, ...).

Approach: Always pick the largest coin $\leq$ remaining amount.

Coins = [1, 5, 10, 25], Amount = 63

Greedy picks: $25 + 25 + 10 + 1 + 1 + 1$
Total coins = 6

• Greedy Template (C++)

```
// Generic Greedy Template
sort(items.begin(), items.end(), compare);
int result = 0;
for (auto &item : items) {
    if (isValid(item)) {
        // take the item
        result += item.value;
        updateState(item);
    }
}
return result;
```

Steps:

1. Sort items based on greedy criterion
2. Iterate and pick the best option
3. Update the state
4. Return answer

• Key Points

- ✔ Works only if greedy choice property holds.
- ✔ Often requires sorting.
- ✔ Gives optimal solution for many real world problems.
- ✔ Not always applicable (e.g. 0/1 Knapsack needs DP).
- ✔ Usually faster and simpler than DP or backtracking.

18. Dynamic Programming Patterns

• What is Dynamic Programming ?

Dynamic Programming (DP) is a technique to solve problems with overlapping subproblems and optimal substructure. Instead of recomputing the same subproblems, we store their results and reuse them.

“Solve once, remember it, and reuse.”

• When to Use DP ?

1. Problem can be broken into smaller subproblems.
2. Subproblems overlap (same subproblem is solved multiple times).
3. Optimal substructure (optimal solution can be built from optimal solutions of subproblems).

• Two Approaches

Top-Down (Memoization)	Bottom-Up (Tabulation)
• Recursive + Cache (DP Array) • Easy to write • Only computes required states	• Iterative • Computes in a specific order • Avoids recursion stack

• Common DP Patterns

Pattern	Typical Problems
1D DP (Linear)	Fibonacci, Climbing Stairs
2D DP (Grid)	Unique Paths, Minimum Path Sum
Subsequence DP	LCS, Edit Distance
Knapsack DP	0/1 Knapsack, Unbounded Knapsack
Partition DP	Subset Sum, Equal Partition
Interval DP	Matrix Chain Multiplication, Burst Balloons
State Machine DP	Buy & Sell Stock, DP on automaton
Digit DP	Count numbers with constraints
Bitmask DP	TSP, Assignment Problems
Tree DP	House Robber III, DP on Trees

• Example 1: Fibonacci (1D DP)

Find the nth Fibonacci number.

```
int fib(int n) {
    if (n <= 1) return n;
    vector<int> dp(n + 1);
    dp[0] = 0; dp[1] = 1;
    for (int i = 2; i <= n; i++)
        dp[i] = dp[i - 1] + dp[i - 2];
    return dp[n];
}
```

Pattern:
1D DP (linear)

Time: $O(n)$
Space: $O(n)$
(Optimizable to $O(1)$)

• Example 2: 0/1 Knapsack

Given n items with weight and value, find the maximum value that fits in a knapsack of capacity W .

```
int knapSack(int n, int W, vector<int>& wt, vector<int>& val) {
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (wt[i - 1] <= w)
                dp[i][w] = max(dp[i - 1][w],
                               val[i - 1] + dp[i - 1][w - wt[i - 1]]);
            else
                dp[i][w] = dp[i - 1][w];
        }
    }
    return dp[n][W];
}
```

Pattern:
Knapsack DP
Time: $O(n \times W)$
Space: $O(n \times W)$

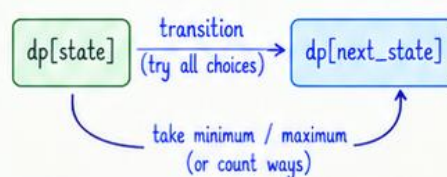
• Example 3: Longest Common Subsequence (LCS)

Given two strings, find the length of their LCS.

```
int lcs(string a, string b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (a[i - 1] == b[j - 1])
                dp[i][j] = 1 + dp[i - 1][j - 1];
            else
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    }
    return dp[n][m];
}
```

Pattern:
Subsequence DP
Time: $O(n \times m)$
Space: $O(n \times m)$

• DP State Transition (General Idea)



Identify:

1. What is the state?
2. How to transition?
3. What is the base case?
4. What is the answer?

• Key Points

- ✓ Use DP when subproblems overlap and optimal substructure exists.
- ✓ Identify the state carefully.
- ✓ Decide on memoization or tabulation.
- ✓ Define base cases clearly.
- ✓ Analyze time and space complexity.
- ✓ Many problems can be converted to DP with the right state definition.

Note :

DP is not a single problem, but a family of patterns. The more you practice, the easier it becomes to recognize the pattern in a new problem.

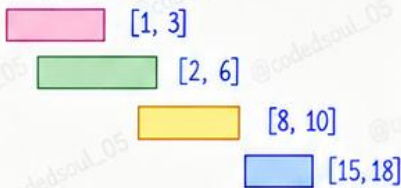
19. Intervals + Bit Manipulation + Trie

1. Intervals

What are Intervals ?

Intervals are ranges, usually represented as [start, end]. Interval problems involve merging, finding overlap, inserting, or scheduling ranges.

Example:



Common Interval Problems

1. Merge Overlapping Intervals
2. Insert Interval
3. Meeting Rooms (minimum rooms)
4. Non-overlapping Intervals
5. Interval Intersection
6. Employee Free Time
7. Minimum Number of Arrows to Burst Balloons
8. Tasks Scheduling

Key Idea

- Sort intervals by start time.
- Use greedy approach (merge/choose).
- Keep track of current interval.

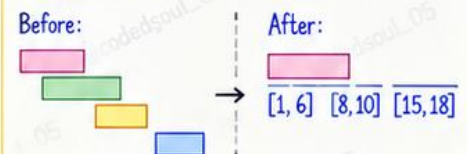
Example: Merge Intervals

Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]

Approach:

1. Sort by start time.
2. Compare current interval with last merged.
3. If overlapping, merge (end = max).
4. Else, add to result.

Visualization:



2. Bit Manipulation

What is Bit Manipulation ?

Bit manipulation means working directly with binary representation of numbers using bitwise operators. It is useful for optimization and solving problems with constraints.

Common Bitwise Operators

&	AND	(both 1 → 1)
	OR	(any 1 → 1)
^	XOR	(different → 1)
~	NOT	(invert bits)
<<	Left Shift	(× 2)
>>	Right Shift	(÷ 2)

Common Problems

1. Find single number (XOR)
2. Find missing number
3. Count set bits
4. Check if number is power of 2
5. Subsets (using bitmask)
6. XOR in a range
7. Find two unique numbers
8. Bit manipulation in DP

Key Tricks

- $n \& (n-1) \rightarrow$ removes last set bit
- $n \& -n \rightarrow$ gets last set bit
- $n \wedge n \rightarrow 0$
- $n \wedge 0 \rightarrow n$
- Use bitmask for subsets/problems
- Check power of 2 : $(n \& (n-1)) == 0$

Example: Single Number

Input: nums = [4, 1, 2, 1, 2]
Output: 4

Approach: XOR all numbers (since $a \wedge a = 0$)

```
int singleNumber(vector<int>& nums) {  
    int x = 0;  
    for (int n : nums) x ^= n;  
    return x;  
}
```

Explanation:

- $4 \wedge 1 \wedge 2 \wedge 1 \wedge 2$
- $(1 \wedge 1) = 0, (2 \wedge 2) = 0$
- Result = 4

XOR helps cancel out duplicate numbers!

3. Trie (Prefix Tree)

What is Trie ?

Trie is a tree-like data structure used to store a collection of strings. It allows fast search, insert, and prefix queries.

Applications

- Dictionary / Autocomplete
- Search suggestions
- Word break problems
- Longest prefix match
- XOR queries (using bitwise trie)
- IP routing (prefix matching)

Trie Node Structure (C++)

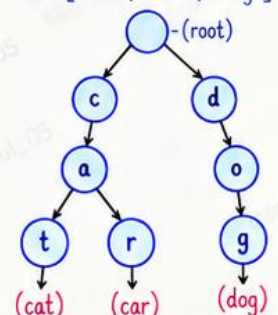
```
struct TrieNode {  
    TrieNode* children[26];  
    bool isEnd;  
    TrieNode() {  
        for (int i = 0; i < 26; i++)  
            children[i] = nullptr;  
        isEnd = false;  
    }  
};
```

Common Problems

1. Insert and Search Word
2. Starts With (prefix search)
3. Word Break
4. Find all words in a board
5. Count distinct substrings
6. Maximum XOR of two numbers
7. Replace words
8. Autocomplete system

Example: Insert & Search

Words: ["cat", "car", "dog"]



Key Points

- Trie is efficient for prefix based problems.
- Time complexity is $O(L)$ per operation (L = length of string).
- Space can be high, but can be optimized (e.g. using hash map instead of fixed array).

"Same patterns, different problems.
Recognize the pattern, solve faster!"

20. DSA Pattern Cheat Sheet + Top Interview Problems

Common DSA Patterns

- Two Pointers
- Sliding Window
- Fast & Slow Pointers
- Prefix Sum
- Hashing / Frequency Count
- Sorting + Greedy
- Binary Search (on answer)
- Stack (Monotonic Stack)
- Queue / Deque (Monotonic Queue)
- Linked List (Reversal, Merge, Cycle)
- Tree Traversals (DFS/BFS)
- Backtracking
- Dynamic Programming
- Graph Traversals (BFS/DFS)
- Shortest Path (Dijkstra, Bellman-Ford)
- Union Find (Disjoint Set Union)
- Intervals (Merge, Insert, Overlap)
- Bit Manipulation
- Trie (Prefix Tree)
- Heap / Priority Queue

Recognize the pattern
→ Apply the template
→ Solve faster !

Pattern → When to Use ?

Pattern	Use When ...
Two Pointers	Pair/Triplet, sorted array, subarray
Sliding Window	Subarray/substring with condition
Fast & Slow	Cycle detection (linked list/array)
Prefix Sum	Range queries, subarray sum
Hashing	Find/Count/Delete in $O(1)$
Sorting + Greedy	Make locally optimal choice
Binary Search	Search in sorted data / answer space
Stack	Next/Previous greater, valid parenthesis
Queue / Deque	Sliding window, BFS, monotonic queue
Linked List	Reverse, merge, detect cycle
Trees	Hierarchical data, traversals, LCA
Backtracking	Try all possibilities, constraints
Dynamic Programming	Overlapping subproblems + optimal
Graphs	Network, connectivity, traversal
Shortest Path	Minimum distance / cost
Union Find	Dynamic connectivity, components
Intervals	Merge, insert, non-overlapping
Bit Manipulation	Optimize space/time, bit operations
Trie	Prefix search, autocomplete
Heap / PQ	Top K, min/max, scheduling

Top Interview Problems (by Topic)

Arrays & Strings

- Two Sum
- Best Time to Buy and Sell Stock
- Longest Substring Without Repeating Characters
- Product of Array Except Self
- Maximum Subarray (Kadane's)
- Find Minimum in Rotated Sorted Array

Linked List

- Reverse Linked List
- Merge Two Sorted Lists
- Linked List Cycle
- Reorder List
- Copy List with Random Pointer

Stack & Queue

- Valid Parentheses
- Next Greater Element
- Daily Temperatures
- Sliding Window Maximum
- Design Circular Queue

Trees

- Inorder / Preorder / Postorder Traversal
- Maximum Depth of Binary Tree
- Validate Binary Search Tree
- Lowest Common Ancestor
- Serialize and Deserialize Binary Tree

Graphs

- Number of Islands
- Clone Graph
- Pacific Atlantic Water Flow
- Course Schedule (Topological Sort)
- Dijkstra's (Network Delay Time)

Heap / Priority Queue

- Kth Largest Element
- Top K Frequent Elements
- Merge K Sorted Lists
- Find Median from Data Stream
- Task Scheduler

Backtracking

- Subsets
- Permutations
- Combination Sum
- Word Search
- N-Queens

Dynamic Programming

- Climbing Stairs
- House Robber
- Longest Increasing Subsequence
- Coin Change
- Edit Distance

Intervals

- Merge Intervals
- Insert Interval
- Non-overlapping Intervals
- Meeting Rooms
- Employee Free Time

Bit Manipulation

- Single Number
- Number of 1 Bits
- Count Bits
- Missing Number
- Reverse Bits

Trie

- Implement Trie (Insert, Search)
- Longest Common Prefix
- Word Search II
- Replace Words
- Design Add and Search Words

Union Find

- Number of Connected Components
- Redundant Connection
- Accounts Merge
- Surrounded Regions
- Most Stones Removed with Same Row/Column

Final Tips

- Identify the pattern before coding
- Write the brute force first, then optimize
- Practice solved problems by patterns
- Do mock interviews and time yourself
- Revise edge cases (empty input, single element, overflow)
- Focus on clarity, not just passing test cases

"Same problems.
Different patterns.
Better you !"

Keep Practicing

- ✓ Solve by pattern
- ✓ Revise regularly
- ✓ Track your progress
- ✓ Do company specific problems (LeetCode, GFG, etc.)
- ✓ Stay consistent !